Curs 4

Programare Orientată pe Obiecte în limbajul Java

Programare Orientată pe Obiecte

Cuprins

- Agregare si mostenire
- Polimorfism
- Blocuri de iniţializare
- Lanţul de apeluri al constructorilor
- Tip - Subtip
- Upcasting/Downcasting
- *Clase abstracte*

Agregare vs. moştenire - *Când se foloseşte moştenirea şi când agregarea?*

- Depinde de datele problemei analizate dar şi de concepţia designerului,
- **Agregarea** - folosită atunci când se doreşte folosirea trăsăturilor unei clase în interiorul altei clase, dar nu şi interfaţa sa (prin moştenire, noua clasă ar expune şi metodele clasei de bază).

Ex: Implementarea funcţionalităţii obiectului conţinut în noua clasă şi **limitarea** acţiunilor utilizatorului la metodele din noua clasă (să nu se permită utilizatorului folosirea metodelor din vechea clasă) - SetAsVector

- noua clasă va conţine un obiect de tipul clasei conţinute, cu specificatorul de acces private.

Agregare vs. moştenire
Când se foloseşte moştenirea şi când compunerea?

- **Moştenirea** permite crearea unor versiuni “specializate” ale unor clase existente (de bază).

- Moştenirea folosită în general atunci când se doreşte construirea unui tip de date care să reprezinte o implementare specifică (o specializare oferită prin clasa derivată) a unui lucru mai general.

Exemplu:

clasa Patrat care moşteneşte clasa Patrulater.

Diferenţa dintre moştenire şi agregare

- Este de fapt diferenţa dintre cele 2 tipuri de relaţii majore prezente între obiectele unei aplicaţii:
- ***is a*** - indică faptul că o clasă este derivată dintr-o clasă de bază

Ex: Având o clasă Animal şi o clasă AnimalDomestic, atunci AnimalDomestic va fi derivat din Animal, cu alte cuvinte AnimalDomestic *is an* Animal

- ***has a*** - indică faptul că o clasă-container are o clasă conţinută în ea

Ex: Dacă avem o clasă Masina şi o clasă Motor, atunci Motor va fi referit în cadrul Masina, cu alte cuvinte Masina *has a* Motor

Problema

Implementati clasa ***SortedArray*** care modeleaza un vector de numere intregi sortat crescator, folosind :

a) mostenire;

b) compunere.

Se va folosi clasa ArrayList (sau Vector).

Se vor defini metodele minime necesare pentru a manipula un astfel de vector.

a)

```
class SortedArray extends Vector{
}
```

b)

```
class SortedArray{
  Vector v;
}
```

Mostenire

```
class SortedArray extends Vector{
  // public SortedArray() { super(); } - implicit
   public boolean add(Object o){
         super.add(o);
         Collections.sort (this);
    }
   public boolean addAll(Collection c){
         super.addAll(c);
         Collections.sort(this);
    }
…..
/* toate add-urile,copyInto, insertElementAt, si
   set-urile*/
}
```

Agregare

```
class SortedArray{
   Vector v;
   public SortedArray() {
      v=new Vector();
   }
   public boolean add(Object o) {
          v.add(o);
          Collections.sort(v);
    }
   public int size() {
        return v.size();
   }
….
/* toate metodele dorite: remove, toString, size etc.. */
}
```

Iniţializare membri de clasă

Blocuri statice de iniţializare

```
static {
    // Bloc static de initializare;
    ...
}
public class Test {
    // Declaratii de variabile statice
    static int x = 0, y, z;
    // Bloc static de initializare
    static {
        System.out.println("Initializam...");
        int t =1;
        y = 2;
        z = x + y + t;
    }
    Test() { ... }
    }
}
```

Blocuri statice de iniţializare

- Variabilele statice ale unei clase sunt iniţializate la un moment care precede prima utilizare activă a clasei respective.
- Momentul efectiv depinde de implementarea maşinii virtuale Java şi poartă numele de iniţializarea clasei. În această etapă sunt executate şi blocurile statice de iniţializare ale clasei.
- Blocurile statice de iniţializare ale clasei se execută în ordinea în care sunt declarate
- Variabilele referite într-un bloc static de iniţializare trebuie să fie obligatoriu de clasă sau locale blocului.

Iniţializare membri de instanţă

- În mod normal în constructor
- Alternativă: blocuri de iniţializare
- La fel ca blocurile statice de iniţializare, dar fără “static”
- Exemplu:

```
{
    //  cod de initializare
}
```

- Compilatorul Java copiază blocurile de iniţializare în fiecare constructor.
- Pot fi folosite pentru un bloc de cod comun mai multor constructori.

Exemplu - blocuri de initializare

```
class Init {
        Init ( int x ) {
            System.out.println("1-arg const");
        }
        Init () {
            System.out.println("no-arg const");
        }
        static {
            System.out.println("1st static init");
        }
        {
            System.out.println("1st instance init");
        }
        {
            System.out.println("2nd instance init");
        }
        static {
            System.out.println("2nd static init");
        }
        public static void main ( String[] args ) {
            new Init();  // new Init(7);
        }
    }
```

Exemplu - blocuri de initializare

Output:

1st static init

2nd static init

1st instance init

2nd instance init

no-arg const

Output:

1st static init

2nd static init

1st instance init

2nd instance init

1-arg const

Lanţul de apeluri al constructorilor

- Cazul în care o clasă de bază este extinsă (moştenită) de către o clasă copil.
- Constructorii nu se moştenesc!
- Constructorii pot fi invocaţi implicit sau explicit (*super*)

- Ori de câte ori este creat un obiect din clasa copil, este invocat mai întâi constructorul fără parametri al clasei părinte.

- Acesta este lanţul implicit de apeluri al constructorilor.

Apelul în lanţ (implicit) al constructorilor

```
class Demo{
   int  value1;
   int  value2;

  Demo() {
      value1 = 1;
      value2 = 2;
      System.out.println("Inside 1st Parent Constructor");
   }
   Demo ( int a ) {
      value1 = a;
      System.out.println("Inside 2nd Parent Constructor");
   }
  public void display(){
     System.out.println("Value1 === "+value1);
     System.out.println("Value2 === "+value2);
  }
}
```

Lanţul de apeluri al constructorilor

```
class DemoChild extends Demo{
    int value3;
    int value4;

   DemoChild() {
          value3 = 3;
          value4 = 4;
         System.out.println("Inside the Constructor of Child");
    }
    public void display(){
          System.out.println("Value1 === "+value1);
           System.out.println("Value2 === "+value2);
      //sau: super.display();
           System.out.println("Value3 === "+value3);
           System.out.println("Value4 === "+value4);
   }
  public static void main(String args[]){
         DemoChild d1 = new DemoChild();
         d1.display();
  }
}
```

Lanţul de apeluri al constructorilor

- În conformitate cu modul implicit de apelare a constructorilor, atunci când este creat un obiect din clasa DemoChild, este apelat mai întâi constructorul Demo() al clasei părinte şi apoi constructorul DemoChild() al clasei copil.

Output :

Inside 1st Parent Constructor

Inside the Constructor of Child

Value1 === 1

Value2 === 2

Value3 === 3

Value4 === 4

Lanţul de apeluri al constructorilor

- Ce se petrece dacă eliminăm constructorul fără parametrii din clasa Demo?
- Dar dacă îi eliminăm pe amândoi?
  - Dacă vrem să apelăm constructorul Demo(int a) în loc de constructorul implicit Demo() atunci când este creat un obiect din clasa copil?
  - Se va folosi cuvântul cheie **“super”** pentru a apela constructorii clasei părinte

Lanţul de apeluri al constructorilor

```
class DemoChild extends Demo{
    int value3;
    int value4;

   DemoChild(){
          super(5);
          value3 = 3;
          value4 = 4;
         System.out.println("Inside the Constructor of Child");
    }

     public void display(){      …   }

     public static void main(String args[]){
               DemoChild d1 = new DemoChild();
              d1.display();
      }
}
```

Output: ?

Lanţul de apeluri al constructorilor

Output:

Inside 2nd Parent Constructor

Inside the Constructor of Child

Value1 === 5

Value2 === 0

Value3 === 3

Value4 === 4

Tip - Subtip

- **int metoda() { return 1.2;} // Eroare**
- **int metoda() { return (int)1.2;} // Corect**
- **double metoda() {return (float)1;} // Corect**

Clasă – Subclasă

Moştenire de tip

- ***O subclasă are acelaşi tip cu superclasa sa, sau, cu alte cuvinte, este de tipul superclasei !!***
- Subclasa moşteneşte reprezentarea şi comportamentul de la superclasă
- Atenţie! NU este valabil invers!

class Patrat extends Poligon { … }

Poligon p = new Patrat(); // corect!

Patrat p = new Poligon(); // incorect!

Exemplu

```
class Patrat extends Poligon { … }

Poligon metoda1( ) {
    Poligon p = new Poligon();
    Patrat t = new Patrat();
    if (...)
        return p; // Corect
    else
        return t; // Corect
}

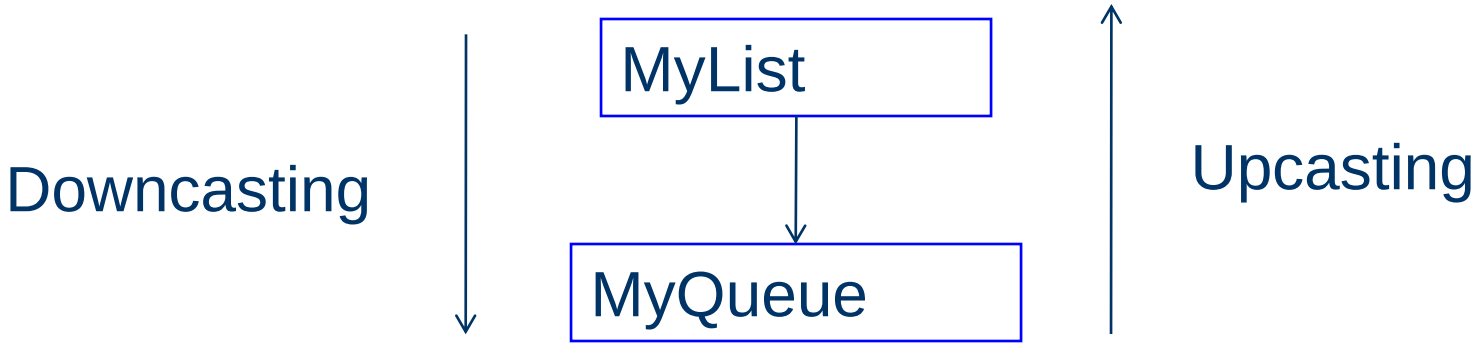
Patrat metoda2( ) {
    Poligon p = new Poligon();
    Patrat t = new Patrat();
    if (...)
        return p; // Eroare
    else
        return t; // Corect
}
```

Upcasting - Downcasting

Programare Orientată pe Obiecte

Upcasting - Downcasting

```
class MyList { }
class MyQueue extends MyList { }
….
MyList a ;
MyQueue q= new MyQueue();
a = q;                          //  upcasting
q = (MyQueue) a ;               //  downcasting
```



Upcasting

- **Convertirea** unei referinţe la o clasă derivată într-una a unei clase de bază
- Realizat **automat** (nu trebuie declarat explicit de către programator)
- Exemplu de upcasting:

```
class Patrat extends Poligon { … }

Poligon metoda1( ) {
    Poligon p = new Poligon();
    Patrat t = new Patrat();
    if (...)  return p; // Corect
    return t; // upcast automat!!! – corect!
}
```

Exemplu

```
class A {... }
class B extends A {… }
public class Test {
    public void method (A a) {
      System.out.println("Method A"); }
    public void method (B b) {
      System.out.println("Method B"); }
    public static void main(String[] args) {
        Test t=new Test();
        B b = new B();
        t.method(b); // "Method B"
        // upcasting a B into an A:
        t.method((A) b); // "Method A"
    }
}
```

Downcasting

- operaţia **inversă** upcast-ului
- conversie explicită de tip în care se merge în **jos** pe ierarhia claselor (se converteşte o clasă de bază într-una derivată)
- trebuie făcut **explicit** de către programator.

```
boolean equals(Object o) {
    if (!(o instanceof Car)) return false;
    Car other = (Car) o;
      // compare this to other and return
}
```

- **posibil** numai dacă obiectul declarat ca fiind de o clasă de bază este, de fapt, **instanţă** a clasei derivate către care se face downcasting-ul. Altfel, maşina virtuală aruncă o excepţie la rularea programului.

Exemplu

```
public class Person {
    private String name;
    private int age;
    public boolean equals (Object anObject) {
        if (anObject == null) return false;
        /* testare daca este de acelasi tip  */
        if (getClass( ) != anObject.getClass())
          return false;
        /* downcast  - Object  clasa parinte a
          oricarei clase */
        Person aPerson = (Person) anObject;
        return name.equals(aPerson.name) &&
          (age == aPerson.age);
    }
}
```

Exemplu

```
public String printAll(LinkedList c) {
    Object arr[ ] = c.toArray();
    String list_string="";
    for(int i=0; i<c.size(); i++) {
        String mn=(String) arr[i];
        list_string+=mn;
    }
    return list_string;
}
```